Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

СЕМИНАР №2
РАБОТА С РАЗРЕЖЕННЫМИ МАТРИЦАМИ

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1 ЦЕЛЬ 3

2 ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ 3

3 РАЗРЕЖЕННЫЕ МАТРИЦЫ 3

4 АЛГОРИТМ 1: COO – КООРДИНАТНЫЙ ФОРМАТ ХРАНЕНИЯ 4

5 АЛГОРИТМ 2: LIL (LINKED LIST MATRIX) – ХРАНЕНИЕ В ФОРМЕ СВЯЗНЫХ СПИСКОВ 6

6 АЛГОРИТМ 3: CSR (COMPRESSED SPARSE ROW) – КОМПАКТНОЕ ХРАНЕНИЕ СИЛЬНО РАЗРЕЖЕННЫХ МАТРИЦ 8

7 ПРИМЕР ПРОГРАММНОЙ РЕАЛИЗАЦИИ 13

1 Цель

Написать консольное приложение для работы с разреженными матрицами.

2 Требования к реализации

Используйте матрицы и зубчатые массивы, условия, циклы, форматированный вывод.

3 Разреженные матрицы

Детальное описание разреженных матриц можно найти здесь – https://ru.wikipedia.org/wiki/Разреженная_матрица

Разреженной называется матрица с преимущественно пустыми (нулевыми) элементами. Нет единого мнения о том какой порог заполненности данных превращает матрицу из плотной (Dense) в разреженную (Sparse). При этом в Data Science существуют задачи, в которых матрица может быть заполнена на 1-5% от общего количества элементов. В этом случае требуются специальные алгоритмы хранения, для того чтобы не хранить 95% нулевых значений.

В настоящее время существуют различные алгоритмы и программные библиотеки для эффективного хранения матриц в разреженном представлении. Одной из наиболее известных является библиотека SciPy – https://docs.scipy.org/doc/scipy/reference/sparse.html

Мы будем решать задачу преобразования матрицы в разреженный формат на основе нескольких алгоритмов.

Иллюстрированное описание алгоритмов приведено в статье – https://matteding.github.io/2019/04/25/sparse-matrices/ Иллюстрации алгоритмов в данном пособии используют рисунки из этой статьи.

4 Алгоритм 1: COO – координатный формат хранения

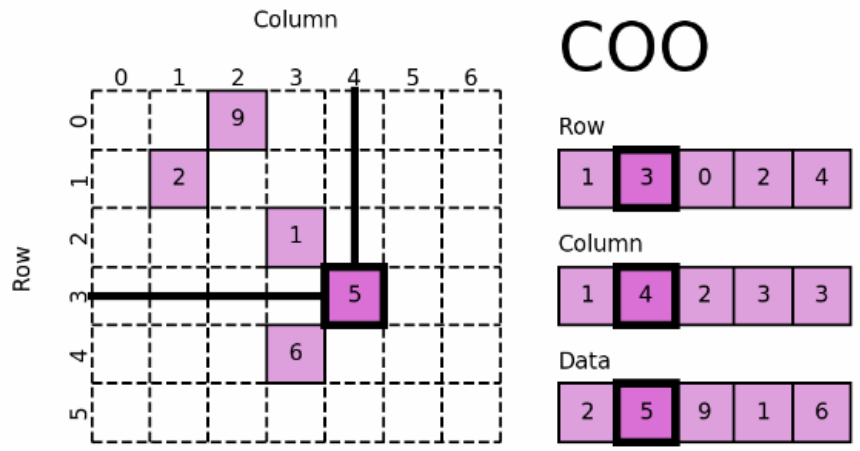

Рис. 1. Принцип работы алгоритма COO (здесь и далее рисунки заимствованы из статьи https://matteding.github.io/2019/04/25/sparse-matrices/).

В случае алгоритма COO плотная матрица заменяется на три одномерных массива:

- массив Row (содержащий строки элементов);
- массив Column (содержащий столбцы элементов);
- массив Data (содержащий значения элементов).

Массивы содержат информацию только о непустых (ненулевых элементах).

Несложно определить порог эффективности хранения элементов в данном формате:

- Пусть дана матрица, содержащая R строк и C столбцов.
- В случае плотного хранения элементы матрицы хранятся последовательно, следовательно требуется R*C ячеек памяти для хранения матрицы.
- В случае разреженного хранения N непустых элементов потребуется N*3 ячеек памяти, так как данные об одном элементе хранятся в 3 массивах.

- Если N*3 = R*C то количество ячеек памяти для плотного и разреженного хранения совпадает.
- Таким образом, для того чтобы разреженное хранение в COO формате было эффективным, необходимо выполнение неравенства для $N < \frac{R*C}{3}$.
- В случае рисунка 1 матрица состоит из 5*6=30 элементов. Поэтому разреженное хранение эффективно только в том случае, когда в матрице хранится менее 10 непустых элементов (где 10 является пороговым значением, при котором эффективность плотного и разреженного хранения совпадает).

Задание 1:

1. Напишите консольное приложение на C# с использованием операторов верхнего уровня и статических функций. Реализуйте следующие функции:
2. Функцию с названием DenseToCOO для преобразования матрицы из плотного формата в разреженный. Функция должна принимать плотную матрицу в качестве входного параметра и возвращать массивы Row, Column, Data как out-параметры.
3. Функцию с названием COOToDense для обратного преобразования матрицы из разреженного формата в плотный. Функция должна принимать на вход массивы Row, Column, Data и возвращать плотную матрицу.
4. Функцию проверки эффективности хранения матрицы в COO формате с названием isCOOEffective. Функция должна учитывать размерности матрицы и ее содержимое (количество непустых элементов). Функция должна принимать на вход плотную матрицу и возвращать логическое значение, которое истинно в случае эффективного хранения.

5. Используйте данные рисунка 1 как тестовый пример и протестируйте работу функций. В программе не используется консольный ввод – матрица, представленная на рисунке 1 задается в тексте программы. Результаты работы функций выводятся с помощью консольного вывода.

5 Алгоритм 2: LIL (Linked List Matrix) – хранение в форме связных списков

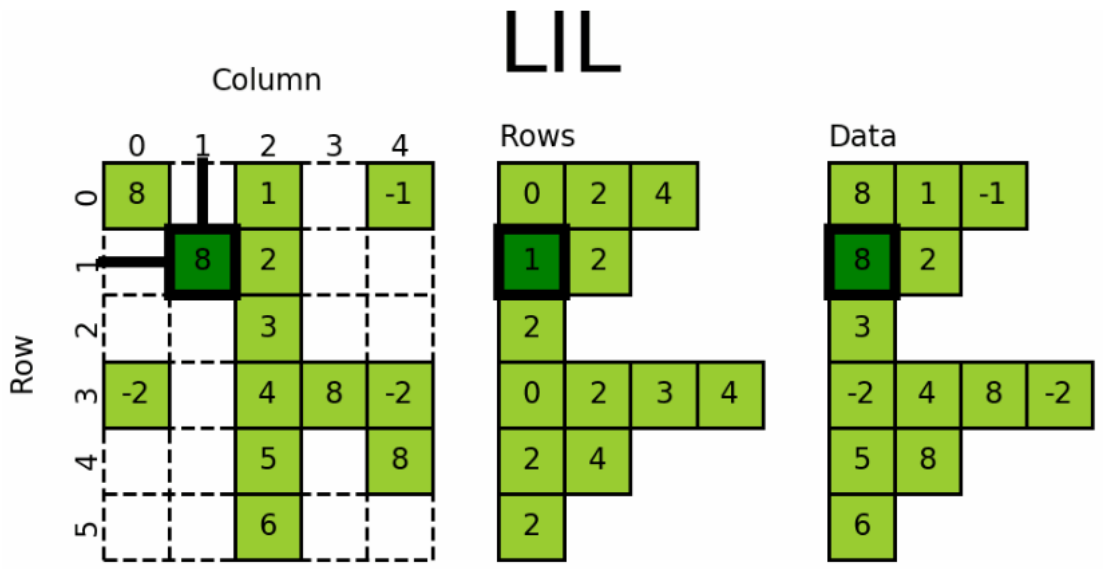

Рис. 2. Принцип работы алгоритма LIL.

В случае алгоритма LIL плотная матрица заменяется на два зубчатых массива:

- массив Rows (содержащий столбцы элементов);
- массив Data (содержащий значения элементов).

Массивы содержат информацию только о непустых (ненулевых элементах).

Задание 2:

1. По аналогии с методом COO определите порог эффективности хранения элементов в формате LIL. Мы исходим из предположения что данные в матрице и зубчатом массиве хранятся

последовательно и размер хранимых данных определяется только количеством выделенных ячеек памяти.

2. Модифицируйте написанное ранее консольное приложение на C#. Дополнительно реализуйте следующие функции:
3. Функцию с названием DenseToLIL для преобразования матрицы из плотного формата в разреженный. Функция должна принимать плотную матрицу в качестве входного параметра и возвращать зубчатые массивы Rows, Data как out-параметры.
4. Функцию с названием LILToDense для обратного преобразования матрицы из разреженного формата в плотный. Функция должна принимать на вход массивы Rows, Data и возвращать плотную матрицу.
5. Функцию проверки эффективности хранения матрицы в LIL формате с названием isLILEffective. Функция должна учитывать размерности матрицы и ее содержимое (количество непустых элементов). Функция должна принимать на вход плотную матрицу и возвращать логическое значение, которое истинно в случае эффективного хранения.
6. Используйте данные рисунка 2 как тестовый пример и протестируйте работу функций. В программе не используется консольный ввод – матрица, представленная на рисунке 2 задается в тексте программы. Результаты работы функций выводятся с помощью консольного вывода.

6 Алгоритм 3: CSR (Compressed Sparse Row) – компактное хранение сильно разреженных матриц

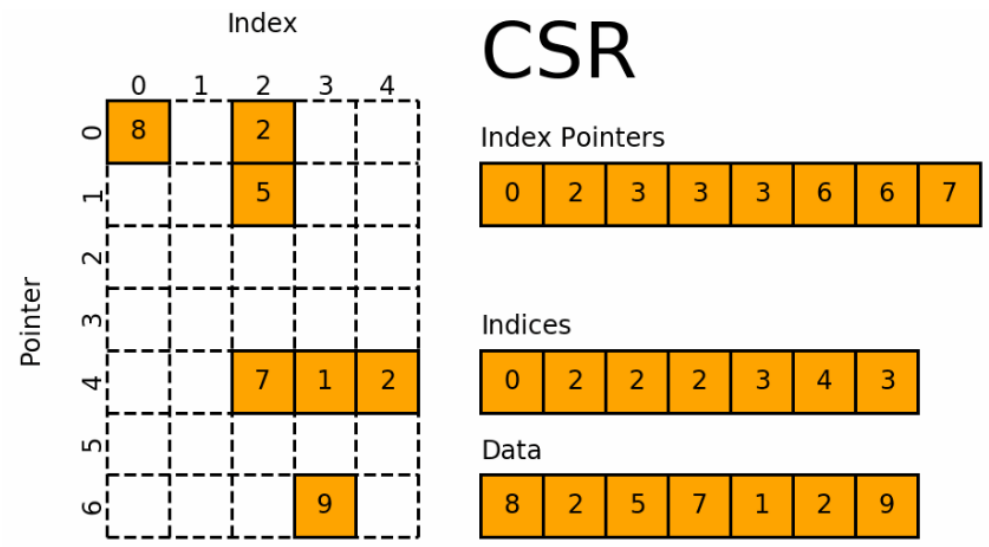

Рис. 3. Принцип работы алгоритма CSR.

Алгоритм CSR отчасти напоминает COO, но принцип хранения более сложный и ориентированный на сильно разреженные матрицы. В случае алгоритма CSR плотная матрица заменяется на три одномерных массива:

- массив Index Pointers (содержащий указатели на начало каждой строки в массиве Data);
- массив Indices (содержащий столбцы элементов);
- массив Data (содержащий значения элементов).

Пошаговый алгоритм кодирования:

ШАГ 1 — Обход матрицы и сбор ненулевых элементов. Проходим матрицу строка за строкой, слева направо. Для каждого ненулевого элемента запоминаем:

- его значение в массиве Data;
- его номер столбца в массиве Indices.

Пример для рисунка 3:

- Row 0: нашли 8 (col=0), нашли 2 (col=2)
- Row 1: нашли 5 (col=2)
- Row 2: пусто
- Row 3: пусто
- Row 4: нашли 7 (col=2), нашли 1 (col=3), нашли 2 (col=4)
- Row 5: пусто
- Row 6: нашли 9 (col=3)

ШАГ 2 — Подсчёт ненулевых элементов в каждой строке.

Считаем, сколько ненулевых элементов содержит каждая строка. Для рисунка 3:

Строка	**Ненулевые элементы**	**Количество ненулевых элементов**
Row 0	8, 2	2
Row 1	5	1
Row 2	нет	0
Row 3	нет	0
Row 4	7, 1, 2	3
Row 5	нет	0
Row 6	9	1

ШАГ 3 — Построение массива Index Pointers.

Index Pointers (IP) — массив из (N строк + 1) элементов.

Каждый элемент показывает, с какого индекса в массиве Data начинается данная строка.

Правило построения:

- IP[0] = 0, массив всегда начинается с 0;

- IP[i+1] = IP[i] + Count(Row i) – предыдущее значение + кол-во непустых элементов в строке i.

Пошаговый пример:

IP[0] = 0 (начало, фиксированное значение)

IP[1] = IP[0] + 2 = 0+2 = 2 (Row 0 содержит 2 элемента)

IP[2] = IP[1] + 1 = 2+1 = 3 (Row 1 содержит 1 элемент)

IP[3] = IP[2] + 0 = 3+0 = 3 (Row 2 содержит 0 элементов)

IP[4] = IP[3] + 0 = 3+0 = 3 (Row 3 содержит 0 элементов)

IP[5] = IP[4] + 3 = 3+3 = 6 (Row 4 содержит 3 элемента)

IP[6] = IP[5] + 0 = 6+0 = 6 (Row 5 содержит 0 элементов)

IP[7] = IP[6] + 1 = 6+1 = 7 (Row 6 содержит 1 элемент)

Последний элемент в массиве IP показывает общее количество ненулевых элементов.

Правила декодирования из формата CSR в плотную матрицу:

- start = IP[i]
- end = IP[i+1]
- Элементы строки i: Data[start : end]
- Столбцы строки i: Indices[start : end]

Примеры:

Пример 1. Чтение Row 4:

- start = IP[4] = 3 (индексация начинается с нуля)
- end = IP[5] = 6
- Data[3:6] = [7, 1, 2]
- Indices[3:6] = [2, 3, 4]

Таким образом:

- matrix[4,2]=7;
- matrix[4,3]=1;
- matrix[4,4]=2.

Пример 2. Чтение Row 2 (пустая строка):

start = IP[2] = 3

end = IP[3] = 3

Data[3:3] = [], нет значений, следовательно вся строка пустая.

Задание 3:

1. По аналогии с методом COO определите порог эффективности хранения элементов в формате CSR. Мы исходим из предположения что данные в матрице и массивах хранятся последовательно и размер хранимых данных определяется только количеством выделенных ячеек памяти.
2. Модифицируйте написанное ранее консольное приложение на C#. Дополнительно реализуйте следующие функции:
3. Функцию с названием DenseToCSR для преобразования матрицы из плотного формата в разреженный. Функция должна принимать плотную матрицу в качестве входного параметра и возвращать массивы Data, Indices, Index Pointers как out-параметры.
4. Функцию с названием CSRToDense для обратного преобразования матрицы из разреженного формата в плотный. Функция должна принимать на вход массивы Data, Indices, Index Pointers и возвращать плотную матрицу.
5. Функцию проверки эффективности хранения матрицы в CSR формате с названием isCSREffective. Функция должна учитывать размерности матрицы и ее содержимое (количество непустых элементов). Функция должна принимать на вход плотную матрицу и возвращать логическое значение, которое истинно в случае эффективного хранения.
6. Используйте данные рисунка 3 как тестовый пример и протестируйте работу функций. В программе не используется

консольный ввод – матрица, представленная на рисунке 3 задается в тексте программы. Результаты работы функций выводятся с помощью консольного вывода.

7 Пример программной реализации

```
// ============================================================
// Структура файла:
//   1. Тестовые матрицы
//   2. Задание 1 — COO
//   3. Задание 2 — LIL
//   4. Задание 3 — CSR
//   5. Вспомогательные статические функции
//   6. Статические функции COO
//   7. Статические функции LIL
//   8. Статические функции CSR
// ============================================================

// ============================================================
//                      ТЕСТОВЫЕ МАТРИЦЫ
// ============================================================

// Тестовая матрица для COO (5 строк × 6 столбцов)
// Всего элементов: 5×6 = 30, ненулевых: 6
// Порог COO: 30/3 = 10 → COO эффективен (6 < 10)
int[,] matrixCOO = {
    { 1, 0, 0, 0, 0, 1 },
    { 0, 2, 0, 0, 0, 0 },
    { 3, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 4 },
    { 0, 0, 5, 0, 0, 0 }
};

// Тестовая матрица для LIL (4 строки × 6 столбцов)
// Всего элементов: 4×6 = 24, ненулевых: 6
// Порог LIL: 24/2 = 12 → LIL эффективен (6 < 12)
int[,] matrixLIL = {
    { 0, 1, 0, 2, 0, 0 },
    { 0, 0, 0, 0, 0, 3 },
    { 4, 0, 5, 0, 0, 0 },
    { 0, 0, 0, 6, 0, 0 }
};

// Тестовая матрица для CSR (7 строк × 5 столбцов)
```

```
// Описана явно в тексте семинара:
//   Row 0: 8 (col=0), 2 (col=2)
//   Row 1: 5 (col=2)
//   Row 2: пусто
//   Row 3: пусто
//   Row 4: 7 (col=2), 1 (col=3), 2 (col=4)
//   Row 5: пусто
//   Row 6: 9 (col=3)
int[,] matrixCSR = {
    { 8, 0, 2, 0, 0 },
    { 0, 0, 5, 0, 0 },
    { 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0 },
    { 0, 0, 7, 1, 2 },
    { 0, 0, 0, 0, 0 },
    { 0, 0, 0, 9, 0 }
};

// ============================================================
//                      ЗАДАНИЕ 1: COO
// ============================================================

PrintSeparator("ЗАДАНИЕ 1: COO - КООРДИНАТНЫЙ ФОРМАТ ХРАНЕНИЯ");

int rCOO = matrixCOO.GetLength(0);
int cCOO = matrixCOO.GetLength(1);
int nCOO = CountNonZero(matrixCOO);

Console.WriteLine("Исходная плотная матрица (Рисунок 1, 5x6):");
PrintMatrix(matrixCOO);

Console.WriteLine("\n--- Анализ эффективности COO ---");
Console.WriteLine($"  Размерность матрицы       : {rCOO} x {cCOO} = {rCOO * cCOO}
ячеек (плотное хранение)");
Console.WriteLine($"  Ненулевых элементов (N)  : {nCOO}");
Console.WriteLine($"  COO хранение (N*3)       : {nCOO}*3 = {nCOO * 3} ячеек");
Console.WriteLine($"  Порог эффективности R*C/3: {rCOO}*{cCOO}/3 = {(double)(rCOO
* cCOO) / 3:F2}");
Console.WriteLine($"  Условие (N*3 < R*C)       : {nCOO * 3} < {rCOO * cCOO} =>
isCOOEffective = {isCOOEffective(matrixCOO)}");
```

```
Console.WriteLine("\n--- Преобразование Dense -> COO ---");
DenseToCOO(matrixCOO, out int[] rowCOO, out int[] colCOO, out int[] dataCOO);
PrintArray("Row   ", rowCOO);
PrintArray("Column", colCOO);
PrintArray("Data  ", dataCOO);

Console.WriteLine("\n--- Преобразование COO -> Dense ---");
int[,] restoredCOO = COOToDense(rowCOO, colCOO, dataCOO, rCOO, cCOO);
PrintMatrix(restoredCOO);
Console.WriteLine($"    Матрица  восстановлена  верно:  {MatricesEqual(matrixCOO,
restoredCOO)}");

// ============================================================
//                      ЗАДАНИЕ 2: LIL
// ============================================================

PrintSeparator("ЗАДАНИЕ 2: LIL - ХРАНЕНИЕ В ФОРМЕ СВЯЗНЫХ СПИСКОВ");

int rLIL = matrixLIL.GetLength(0);
int cLIL = matrixLIL.GetLength(1);
int nLIL = CountNonZero(matrixLIL);

Console.WriteLine("Исходная плотная матрица (Рисунок 2, 4x6):");
PrintMatrix(matrixLIL);

Console.WriteLine("\n--- Анализ эффективности LIL ---");
Console.WriteLine($"  Размерность матрицы       : {rLIL} x {cLIL} = {rLIL * cLIL}
ячеек (плотное хранение)");
Console.WriteLine($"  Ненулевых элементов (N)  : {nLIL}");
Console.WriteLine($"  LIL хранение (N*2)       : {nLIL}*2 = {nLIL * 2} ячеек");
Console.WriteLine($"  Порог эффективности R*C/2: {rLIL}*{cLIL}/2 = {(double)(rLIL
* cLIL) / 2:F2}");
Console.WriteLine($"  Условие (N*2 < R*C)       : {nLIL * 2} < {rLIL * cLIL} =>
isLILEffective = {isLILEffective(matrixLIL)}");

Console.WriteLine("\n--- Преобразование Dense -> LIL ---");
DenseToLIL(matrixLIL, out int[][] rowsLIL, out int[][] dataLIL);
Console.WriteLine("  Rows[i] — индексы столбцов ненулевых элементов строки i:");
PrintJaggedArray("Rows", rowsLIL);
```

```
Console.WriteLine("  Data[i] — значения ненулевых элементов строки i:");
PrintJaggedArray("Data", dataLIL);

Console.WriteLine("\n--- Преобразование LIL -> Dense ---");
int[,] restoredLIL = LILToDense(rowsLIL, dataLIL, rLIL, cLIL);
PrintMatrix(restoredLIL);
Console.WriteLine($"    Матрица  восстановлена  верно:  {MatricesEqual(matrixLIL,
restoredLIL)}");

// ============================================================
//                      ЗАДАНИЕ 3: CSR
// ============================================================

PrintSeparator("ЗАДАНИЕ 3: CSR - КОМПАКТНОЕ ХРАНЕНИЕ РАЗРЕЖЕННЫХ МАТРИЦ");

int rCSR = matrixCSR.GetLength(0);
int cCSR = matrixCSR.GetLength(1);
int nCSR = CountNonZero(matrixCSR);

Console.WriteLine("Исходная плотная матрица (Рисунок 3, 7x5):");
PrintMatrix(matrixCSR);

Console.WriteLine("\n--- Анализ эффективности CSR ---");
Console.WriteLine($"  Размерность матрицы             : {rCSR} x {cCSR} = {rCSR *
cCSR} ячеек (плотное хранение)");
Console.WriteLine($"  Ненулевых элементов (N)        : {nCSR}");
Console.WriteLine($"  CSR хранение (2*N + R + 1)      : 2*{nCSR} + {rCSR} + 1 = {2
* nCSR + rCSR + 1} ячеек");
Console.WriteLine($"  Порог: (R*C - R - 1) / 2        : ({rCSR * cCSR} - {rCSR} -
1) / 2 = {(double)(rCSR * cCSR - rCSR - 1) / 2:F2}");
Console.WriteLine($"  Условие (2N+R+1 < R*C)           : {2 * nCSR + rCSR + 1} <
{rCSR * cCSR} => isCSREffective = {isCSREffective(matrixCSR)}");

Console.WriteLine("\n--- Преобразование Dense -> CSR ---");
DenseToCSR(matrixCSR, out int[] dataCSR, out int[] indicesCSR, out int[] ipCSR);
PrintArray("Data         ", dataCSR);
PrintArray("Indices      ", indicesCSR);
PrintArray("IndexPointers", ipCSR);

Console.WriteLine("\n--- Пошаговое декодирование строк из CSR ---");
```

```
for (int i = 0; i < rCSR; i++)
{
    int start = ipCSR[i];
    int end = ipCSR[i + 1];
    Console.Write($"  Row {i}: IP[{i}]={start}, IP[{i + 1}]={end} => ");
    if (start == end)
    {
        Console.WriteLine("(пустая строка)");
    }
    else
    {
        for (int k = start; k < end; k++)
            Console.Write($"matrix[{i},{indicesCSR[k]}]={dataCSR[k]}  ");
        Console.WriteLine();
    }
}

Console.WriteLine("\n--- Преобразование CSR -> Dense ---");
int[,] restoredCSR = CSRToDense(dataCSR, indicesCSR, ipCSR, rCSR, cCSR);
PrintMatrix(restoredCSR);
Console.WriteLine($"    Матрица  восстановлена  верно:  {MatricesEqual(matrixCSR,
restoredCSR)}");

Console.WriteLine("\n=== ПРОГРАММА ЗАВЕРШЕНА ===");

// ============================================================
//                  ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
// ============================================================

// Выводит заголовок раздела в рамке.
static void PrintSeparator(string title)
{
    string line = new string('=', 62);
    Console.WriteLine($"\n{line}");
    Console.WriteLine($"  {title}");
    Console.WriteLine($"{line}\n");
}

// Выводит двумерную матрицу на консоль с выравниванием.
static void PrintMatrix(int[,] m)
```

```
{
    int rows = m.GetLength(0);
    int cols = m.GetLength(1);
    for (int i = 0; i < rows; i++)
    {
        Console.Write("  |");
        for (int j = 0; j < cols; j++)
            Console.Write($" {m[i, j],3}");
        Console.WriteLine(" |");
    }
}

// Выводит одномерный массив на консоль.
static void PrintArray(string name, int[] arr)
{
    Console.Write($"  {name}: [ ");
    for (int i = 0; i < arr.Length; i++)
        Console.Write($"{arr[i],3}{(i < arr.Length - 1 ? "," : " ")}");
    Console.WriteLine("]");
}

// Выводит зубчатый массив на консоль — строка за строкой.
static void PrintJaggedArray(string name, int[][] arr)
{
    Console.WriteLine($"  {name}:");
    for (int i = 0; i < arr.Length; i++)
    {
        Console.Write($"    [{i}]: [ ");
        if (arr[i].Length == 0)
        {
            Console.WriteLine("(пусто) ]");
        }
        else
        {
            for (int k = 0; k < arr[i].Length; k++)
                Console.Write($"{arr[i][k],3}{(k < arr[i].Length - 1 ? "," : "
")}");
            Console.WriteLine("]");
        }
    }
```

```
}

// Подсчитывает количество ненулевых элементов матрицы.
static int CountNonZero(int[,] m)
{
    int count = 0;
    for (int i = 0; i < m.GetLength(0); i++)
        for (int j = 0; j < m.GetLength(1); j++)
            if (m[i, j] != 0)
                count++;
    return count;
}

// Сравнивает две матрицы поэлементно. Возвращает true если матрицы равны.
static bool MatricesEqual(int[,] a, int[,] b)
{
    if (a.GetLength(0) != b.GetLength(0) || a.GetLength(1) != b.GetLength(1))
        return false;
    for (int i = 0; i < a.GetLength(0); i++)
        for (int j = 0; j < a.GetLength(1); j++)
            if (a[i, j] != b[i, j])
                return false;
    return true;
}

// ============================================================
//              СТАТИЧЕСКИЕ ФУНКЦИИ — АЛГОРИТМ COO
// ============================================================

// Преобразует плотную матрицу в COO-формат.
// Обходит матрицу строка за строкой.
// Для каждого ненулевого элемента записывает в три массива:
//   row[k]  — номер строки элемента
//   col[k]  — номер столбца элемента
//   data[k] — значение элемента
static void DenseToCOO(int[,] dense,
                       out int[] row,
                       out int[] col,
                       out int[] data)
{
```

```
    int rows = dense.GetLength(0);
    int cols = dense.GetLength(1);
    int count = CountNonZero(dense);

    row = new int[count];
    col = new int[count];
    data = new int[count];

    int idx = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (dense[i, j] != 0)
            {
                row[idx] = i;
                col[idx] = j;
                data[idx] = dense[i, j];
                idx++;
            }
}

// Восстанавливает плотную матрицу из COO-формата.
// Создаёт нулевую матрицу нужного размера и расставляет
// элементы по координатам из массивов row и col.
// Параметры numRows и numCols задают размер исходной матрицы
// (необходимы, т.к. последние строки/столбцы могут быть нулевыми).
static int[,] COOToDense(int[] row, int[] col, int[] data,
                         int numRows, int numCols)
{
    int[,] dense = new int[numRows, numCols]; // по умолчанию заполнена нулями
    for (int k = 0; k < data.Length; k++)
        dense[row[k], col[k]] = data[k];
    return dense;
}

// Проверяет, эффективно ли хранение матрицы в COO-формате.
// COO хранит каждый элемент в 3 ячейках (row + col + data).
// Условие эффективности: N*3 < R*C, т.е. N < R*C/3
static bool isCOOEffective(int[,] dense)
{
    int r = dense.GetLength(0);
```

```
    int c = dense.GetLength(1);
    int n = CountNonZero(dense);
    // Плотное хранение: R*C ячеек
    // COO хранение:     N*3 ячеек
    // Условие эффективности: N*3 < R*C
    return n * 3 < r * c;
}

// ============================================================
//              СТАТИЧЕСКИЕ ФУНКЦИИ — АЛГОРИТМ LIL
// ============================================================

// Преобразует плотную матрицу в LIL-формат (два зубчатых массива).
// Для каждой строки i:
//   rows[i] — подмассив с индексами СТОЛБЦОВ ненулевых элементов строки i
//   data[i] — подмассив со значениями  ненулевых элементов строки i
// Внимание: rows[i] хранит индексы столбцов, а не строк!
static void DenseToLIL(int[,] dense,
                       out int[][] rows,
                       out int[][] data)
{
    int numRows = dense.GetLength(0);
    int numCols = dense.GetLength(1);

    rows = new int[numRows][];
    data = new int[numRows][];

    for (int i = 0; i < numRows; i++)
    {
        // Шаг 1: подсчёт ненулевых элементов в строке i
        int count = 0;
        for (int j = 0; j < numCols; j++)
            if (dense[i, j] != 0)
                count++;

        // Шаг 2: выделяем подмассивы точного нужного размера
        rows[i] = new int[count]; // индексы столбцов (не строк!)
        data[i] = new int[count]; // значения элементов

        // Шаг 3: заполняем подмассивы
```

```
        int idx = 0;
        for (int j = 0; j < numCols; j++)
            if (dense[i, j] != 0)
            {
                rows[i][idx] = j;           // запоминаем номер столбца
                data[i][idx] = dense[i, j]; // запоминаем значение
                idx++;
            }
    }
}

// Восстанавливает плотную матрицу из LIL-формата.
// Для каждой строки i берёт пары (rows[i][k], data[i][k])
// и записывает значение в dense[i, rows[i][k]].
static int[,] LILToDense(int[][] rows, int[][] data,
                         int numRows, int numCols)
{
    int[,] dense = new int[numRows, numCols]; // по умолчанию заполнена нулями
    for (int i = 0; i < numRows; i++)
        for (int k = 0; k < rows[i].Length; k++)
            dense[i, rows[i][k]] = data[i][k];
    return dense;
}

// Проверяет, эффективно ли хранение матрицы в LIL-формате.
// LIL хранит каждый элемент в 2 ячейках (rows + data).
// Условие эффективности: N*2 < R*C, т.е. N < R*C/2
static bool isLILEffective(int[,] dense)
{
    int r = dense.GetLength(0);
    int c = dense.GetLength(1);
    int n = CountNonZero(dense);
    // Плотное хранение: R*C ячеек
    // LIL хранение:     N*2 ячеек
    // Условие эффективности: N*2 < R*C
    return n * 2 < r * c;
}

// ============================================================
//              СТАТИЧЕСКИЕ ФУНКЦИИ — АЛГОРИТМ CSR
```

```
// ============================================================

// Преобразует плотную матрицу в CSR-формат (три одномерных массива).
//
// ШАГ 1: обход матрицы строка за строкой.
//   Для каждого ненулевого элемента:
//     data[idx]    = значение элемента
//     indices[idx] = номер столбца элемента
//
// ШАГ 2: построение массива IndexPointers (IP).
//   IP[i]      = начальный индекс строки i в массивах data и indices.
//   IP[0]      = 0 (всегда)
//   IP[i+1]    = IP[i] + кол-во ненулевых элементов в строке i
//   IP[numRows] = N (общее кол-во ненулевых — последний элемент)
static void DenseToCSR(int[,] dense,
                       out int[] data,
                       out int[] indices,
                       out int[] indexPointers)
{
    int numRows = dense.GetLength(0);
    int numCols = dense.GetLength(1);
    int n = CountNonZero(dense);

    data = new int[n];
    indices = new int[n];
    indexPointers = new int[numRows + 1]; // размер всегда R+1

    int idx = 0;
    for (int i = 0; i < numRows; i++)
    {
        indexPointers[i] = idx; // IP[i] = начало строки i
        for (int j = 0; j < numCols; j++)
            if (dense[i, j] != 0)
            {
                data[idx] = dense[i, j]; // значение
                indices[idx] = j;           // номер столбца
                idx++;
            }
    }
    indexPointers[numRows] = n; // последний элемент = общее кол-во ненулевых
```

```
}

// Восстанавливает плотную матрицу из CSR-формата.
// Правила декодирования для строки i:
//   start = IP[i]
//   end   = IP[i+1]
//   элементы строки i: Data[start .. end-1]
//   столбцы строки  i: Indices[start .. end-1]
// Если start == end — строка пустая (нет ненулевых элементов).
static int[,] CSRToDense(int[] data, int[] indices, int[] indexPointers,
                         int numRows, int numCols)
{
    int[,] dense = new int[numRows, numCols]; // по умолчанию заполнена нулями
    for (int i = 0; i < numRows; i++)
    {
        int start = indexPointers[i];
        int end = indexPointers[i + 1];
        // Читаем только элементы строки i (диапазон [start, end))
        for (int k = start; k < end; k++)
            dense[i, indices[k]] = data[k];
    }
    return dense;
}

// Проверяет, эффективно ли хранение матрицы в CSR-формате.
// CSR хранит: N (Data) + N (Indices) + R+1 (IndexPointers) = 2*N + R + 1 ячеек.
// Условие эффективности: 2*N + R + 1 < R*C
static bool isCSREffective(int[,] dense)
{
    int r = dense.GetLength(0);
    int c = dense.GetLength(1);
    int n = CountNonZero(dense);
    // Плотное хранение: R*C ячеек
    // CSR хранение:     2*N + R + 1 ячеек
    // Условие эффективности: 2*N + R + 1 < R*C
    return 2 * n + r + 1 < r * c;
}
```